

MÓDULO 2 · PYTHON PARA ANÁLISIS DE DATOS — CLASE 4

Tu código toma decisiones

Condicionales y ciclos

Bootcamp de Ciencia de Datos · Estructuras de control: condicionales y ciclos

BIENVENIDA

El mapa del bootcamp: 6 módulos, 32 clases

MÓDULO 1**Fundamentos y Mentalidad de Datos**

Completado

MÓDULO 2**Python para Análisis de Datos**

Estamos aquí

MÓDULO 3**Visualización y Business Intelligence**

Gráficas, EDA, Power BI

MÓDULO 4**Extracción de Datos**

Bases de datos, SQL, APIs

MÓDULO 5**Machine Learning Básico y Aplicado**

Clasificación, regresión, clustering

MÓDULO 6**Proyecto Final y Cierre**

Deep Learning, NLP, tu proyecto

32 CLASES**Tu recorrido completo**

Mayo a septiembre, 2 clases por semana

CÓMO TRABAJAMOS

Teoría + práctica en cada módulo · talleres integradores · proyecto final

Hoy tu código deja de ejecutarse en línea recta: aprende a decidir y a repetir, dos superpoderes de cualquier programa.

POR QUÉ HOY IMPORTA

De 10.000 transacciones a días de trabajo... o segundos

Condicionales + Ciclos

Cómo se le da "inteligencia" y "velocidad" a un programa

- Imagina que te entregan un archivo con 10.000 transacciones y debes aplicar un 15% de descuento solo a las que superen \$500.
- Hacerlo a mano te tomaría días, con un margen de error altísimo.
- Un condicional le enseña a Python a decidir; un ciclo le enseña a repetir esa decisión miles de veces.
- Juntos, resuelven en segundos lo que a una persona le tomaría días.

¿Por qué justo ahora?



Ya sabes guardar datos

Variables, listas, diccionarios: ya tienes dónde guardar información. Hoy aprendes a actuar sobre ella.



Es la base de cualquier lógica de negocio

Descuentos, alertas, validaciones: casi toda regla de negocio se traduce en un if, un for o un while.



Prepara el terreno para Pandas

Cuando limpies datos reales en la Clase 7, vas a recorrer y filtrar filas exactamente con esta misma lógica.

Al final de la clase, tu código va a decidir y repetir tareas automáticamente, sin que tú tengas que hacerlo a mano.

CLASE 4

Lo que vamos a lograr hoy



01

Tomar decisiones con condicionales

Usar if, elif y else, incluso combinando varias condiciones a la vez.



02

Repetir tareas con ciclos

Usar for cuando sabes cuántas veces repetir, y while cuando dependes de una condición.



03

Controlar el flujo de un ciclo

Usar break y continue para saltar o detener un ciclo antes de que termine solo.

Al final de la clase: un programa que clasifica datos automáticamente, sin revisarlos uno por uno.

MÓDULO 2 · CONDICIONALES

Entonces, ¿qué son los condicionales?

Le indican a Python cómo tomar decisiones basadas en condiciones. Python revisa cada una de arriba hacia abajo, y se detiene en la primera que sea verdadera.



if

Si esta condición es verdadera, ejecuta este bloque.



elif

Si la anterior no se cumplió, prueba con esta otra.



else

Si ninguna se cumplió, ejecuta esto por defecto.



El orden importa

Cambiar el orden de las condiciones puede cambiar por completo el resultado.

Chequeo rápido: si un cliente compra por \$600 y tiene tarjeta de puntos, ¿cuántas condiciones distintas necesitarías evaluar?

MÓDULO 2 · CICLOS

Dos ciclos, dos propósitos

**for**

Cuando sabes qué recorrer.

- Recorre una colección completa
- Una vuelta por cada elemento
- Termina cuando se acaban los elementos

Línea de ensamblaje

**while**

Cuando dependes de una condición.

- Repite mientras algo sea cierto
- No sabes cuántas vueltas dará
- Riesgo de ciclo infinito si no cambia nada

Auto acelerando

**break / continue**

Para controlar cualquier ciclo.

- continue salta la vuelta actual
- break termina el ciclo completo
- Funcionan en for y en while

Control fino

La pregunta que los distingue: ¿ya sabes cuántas veces vas a repetir, o depende de una condición que cambia?

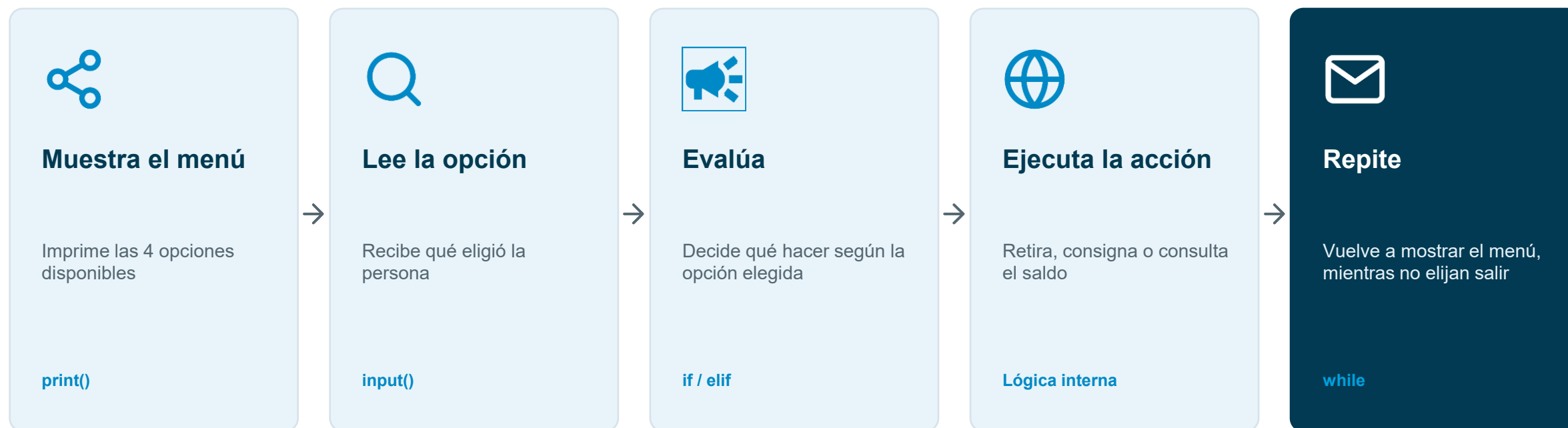
MÓDULO 2 · ESTRUCTURAS DE CONTROL

Cómo se conectan las piezas



MÓDULO 2 · CICLOS

El menú del cajero, paso a paso



La idea clave: el programa no sabe cuántas veces va a repetir el menú. Por eso es un `while`, no un `for`: depende de una condición que cambia con cada decisión de la persona.

MÓDULO 2 · CICLOS

Entonces, ¿qué es un ciclo while?



Repite mientras algo sea verdadero

A diferencia de un for, que sabe exactamente cuántos elementos tiene una lista, un while se repite una y otra vez mientras una condición se mantenga verdadera, sin saber de antemano cuántas vueltas dará.

Es ideal para metas, validaciones y menús interactivos.

Un ejemplo aplicado

Repetir una venta de \$250 hasta alcanzar una meta de \$1000, contando cuántos clientes hicieron falta.

Dato: si la condición de un while nunca deja de cumplirse, el ciclo nunca termina. Siempre confirma que algo dentro del ciclo cambie el valor evaluado.

Actividad: bonos y clasificación de edades

1

Con la lista de calificaciones [5,2,8,1,5,9,10,10,3,4,8], usa for + if/else: 8 o más recibe bono, menos de 8 recibe capacitación.

8 min

2

Con la lista de edades [12,17,8,15,22,9,30,25], recorre con for: si la edad es 25, imprime "¡Encontramos al estudiante de 25 años! Deteniendo el análisis." y usa break.

7 min

3

En el mismo ciclo: si la edad es menor a 10, sáltala con continue.

5 min

4

Para el resto: 18 o más imprime "Adulto: [edad] años", si no, "Menor: [edad] años".

5 min



Entregable

Una captura de pantalla con:

- Los mensajes de bono/capacitación por empleado
- El break activado al llegar a 25 años
- Los mensajes de Adulto/Menor para el resto

Este patrón (for + if/else, con break y continue) lo vas a usar constantemente al limpiar datos reales.

Si tu ciclo nunca termina, revisa que la condición del while sí pueda volverse falsa en algún momento.

CIERRE DE ACTIVIDAD · 10 MIN

Puesta en común: revisemos los resultados



Algunos voluntarios comparten

Cada quien muestra su código y los mensajes impresos.



Los demás retan

¿El break se activó exactamente al llegar a 25? ¿El continue saltó bien los menores de 10?



Buscamos patrones

¿A todos les quedó el mismo orden de condiciones? ¿Alguien resolvió el ciclo de forma distinta?

Pregunta de cierre: ¿en qué otra situación de tu trabajo usarías un ciclo que se detenga apenas encuentre algo específico?

RECAPITULEMOS

Las 4 ideas que se llevan hoy

01**Python revisa condiciones de arriba hacia abajo**

Se detiene en la primera que sea verdadera; el orden puede cambiar el resultado.

02**for cuando sabes qué recorrer, while cuando depende de una condición**

Elegir el ciclo correcto evita errores y ciclos infinitos.

03**break detiene, continue salta**

Ambos te dan control fino sobre cualquier ciclo, sea for o while.

04**Ya automatizaste una decisión repetitiva**

Lo que antes tomaría revisar dato por dato, ahora lo resuelve un ciclo en segundos.

Mini-reto de salida (1 frase): explícale a alguien la diferencia entre break y continue, sin usar la palabra "ciclo".